



**COMPOSABLE
SECURITY**

REPORT

Security consultation for Lido

Prepared by: Composable Security

Report ID: LDO-0fb275b7

Test time period: 2026-07-10 - 2026-07-10

Report date: 2026-07-10

Version: 1.0

Visit: composable-security.com

Contents

1. Security consultation summary (2026-07-10)	2
1.1 Subject of Consultation	2
1.2 Scope	2
1.3 Root Cause	2
1.4 The Fix	2
1.5 Consultation Results	3
1.6 Deployments	3
1.7 Disclaimer	3

1. Security consultation summary (2026-07-10)

1.1. Subject of Consultation

The **Composable Security** team was engaged by **Lido** to perform a security review of a hot fix update of the Lido Oracle, including:

- fix for an issue in CL rebase calculation that incorrectly calculated the injected deposits,
- adding a check that distant/nearest slots are within the frame,
- comment with clarification of the security posture of the performance-web sidecar.

This fix addresses an issue that led to an invalid value of deposits injected within a window between the nearest/distant slot and the ref slot of a report.

1.2. Scope

The assessment focused on changes introduced in version **8.0.2**.

GitHub repository:

<https://github.com/lidofinance/lido-oracle>

CommitID: 0fb275b7a3030ffed36a64182b3d91cd2de6bfc6

1.3. Root Cause

The Lido Oracle previously calculated the value of injected deposits between two slots as the difference between the `deposited_for_current_report` values queried on-chain in the before-mentioned slots.

The issue is that this value includes deposits from the previous frame at slots earlier than the report settlement slot.

1.4. The Fix

The initial solution used the following formula to calculate the deposits injected in the window defined by slots:

```
1 injected_deposits(prev_slot -> ref_slot) =  
2     deposited_since_last_report(ref_slot) -  
3     (deposited_since_last_report(prev_slot) - deposited_for_current_report(  
        prev_slot))
```

It correctly subtracted any deposits from previous frame for `prev_slots` that are earlier than the report settlement slot.

After consultation, the formula has been updated to:

```

1 injected_deposits(prev_slot -> ref_slot) =
2   (deposited_since_last_report(ref_slot) - deposited_for_current_report(
3     ref_slot)) -
4   (deposited_since_last_report(prev_slot) - deposited_for_current_report(
5     prev_slot))

```

It subtracts any deposits from previous frames for `ref_slot` if there was a report missing.

1.5. Consultation Results

After consultation, the implemented fix effectively resolves the injected deposits calculation issue, as it:

1. subtracts the value of `deposited_for_current_report` from `deposited_since_last_report` for the `prev_slot` to ignore any deposits from previous frame in case the report has not been settled yet (those deposits are present in both of those values),
2. subtracts the value of `deposited_for_current_report` from `deposited_since_last_report` for the `ref_slot` to ignore any deposits from previous frames in case there was a missing report,
3. subtracts the first value from second to get the value of deposits made between `prev_slot` and `ref_slot`.

It is important to mention that the previous slot must be within the same frame as the reference slot, but such checks have also been added.

1.6. Deployments

After the security review conducted, we verified that the Dockerfile used to build an image uses the reviewed source code. The published image with the manifest digest **sha256:606aa5d3c05a9135e5be1e589229ba6e22d941aa586779af0e58a9dd811a93ce** corresponds to the commitID: **0fb275b7a3030ffed36a64182b3d91cd2de6bfc6**.

1.7. Disclaimer

Security consultation **IS NOT A SECURITY WARRANTY**.

During the review, the Composable Security team makes every effort to detect any occurring problems and help to address them. However, it is not allowed to treat the report as a security certificate and assume that the project does not contain any vulnerabilities. Securing applications is a multi-stage process.

Therefore, we encourage the implementation of security mechanisms at all stages of development and maintenance.



Damian Rusinek

Smart Contracts Auditor

@drdr_zz

damian.rusinek@composable-security.com



Paweł Kuryłowicz

Smart Contracts Auditor

@wh01s7

pawel.kurylowicz@composable-security.com

